

Module 09 Lab – Ensemble Methods

Objective: To learn how to use **ensemble methods**, which combine the predictions of several individual models to produce a final prediction that is more accurate and robust than any of the individual models alone.**In this lab, you will train and compare a single Decision Tree against a powerful ensemble method, the Random Forest.**

Part 1: The Wisdom of the Crowd

Concept: The core idea behind ensemble methods is the "wisdom of the crowd." If you ask one person a difficult question, they might be wrong. But if you ask a large, diverse group of people the same question and average their answers, the result is often surprisingly accurate. Ensemble methods apply this to machine learning. Instead of relying on a single model, we build a "committee" of models and have them vote on the final prediction. Two popular ensemble techniques:

- * **Bagging (Bootstrap Aggregating):** Trains multiple models (e.g., Decision Trees) on different random subsets of the training data. The **Random Forest** is the most famous example.
- * **Boosting:** Trains models sequentially, where each new model focuses on correcting the errors made by the previous one. Examples include Gradient Boosting and AdaBoost.

Part 2: Setup

We will use the Iris dataset to classify flower species.

```
1 import pandas as pd
2 from sklearn.datasets import load_iris
3 from sklearn.model_selection import train_test_split
4 from sklearn.tree import DecisionTreeClassifier
5 from sklearn.ensemble import RandomForestClassifier
6 from sklearn.metrics import accuracy_score
7 # Load and prepare data
8 iris = load_iris()
9 X = iris.data
10 y = iris.target
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Part 3: A Single Decision Tree

A single Decision Tree is prone to overfitting. It can create very specific rules that match the training data perfectly but don't generalize well.

Your Task: Train a single `DecisionTreeClassifier` and evaluate its accuracy.

```
1 # --- ENTER YOUR CODE HERE ---
2 # 1. Create and train a single Decision Tree
3 single_tree = DecisionTreeClassifier(random_state=42)
4 single_tree.fit(X_train, y_train)
5 # 2. Make predictions and evaluate its accuracy
6 y_pred_tree = single_tree.predict(X_test)
7 accuracy_tree = accuracy_score(y_test, y_pred_tree)
8 print(f"Accuracy of a single Decision Tree: {accuracy_tree:.2%}")
```

Accuracy of a single Decision Tree: 100.00%

Part 4: The Random Forest Ensemble

Concept: A Random Forest is an ensemble of many Decision Trees. It builds hundreds of trees, each trained on a random subset of the data and using a random subset of the features. To make a prediction, it gets a vote from every tree in the "forest" and chooses the class that gets the most votes. This process of averaging the votes of many diverse trees makes the Random Forest much more robust and less prone to overfitting than a single tree.

Your Task: Train a `RandomForestClassifier` and compare its accuracy to the single tree.

```
1 # --- ENTER YOUR CODE HERE ---
2 # 1. Create and train a Random Forest
3 # `n_estimators` is the number of trees in the forest.
4 random_forest = RandomForestClassifier(n_estimators=100, random_state=42)
5 random_forest.fit(X_train, y_train)
6 # 2. Make predictions and evaluate its accuracy
7 y_pred_rf = random_forest.predict(X_test)
8 accuracy_rf = accuracy_score(y_test, y_pred_rf)
9 print(f"Accuracy of the Random Forest: {accuracy_rf:.2%}")
```

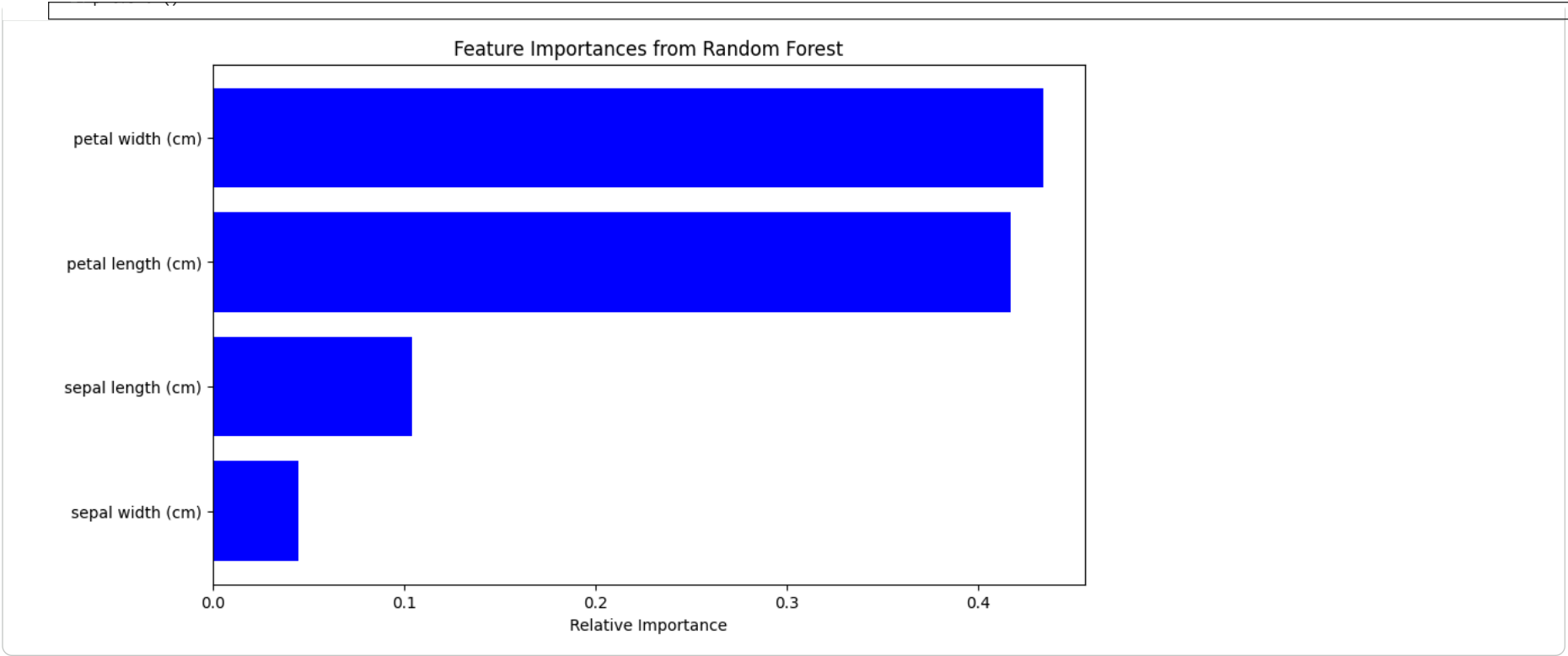
Accuracy of the Random Forest: 100.00%

Part 5: Feature Importance

Concept: A great benefit of tree-based ensembles like Random Forest is that they can tell you which features were most important for making accurate predictions.

Your Task: Extract and plot the feature importances from your trained Random Forest model.

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 # --- ENTER YOUR CODE HERE ---
4 # 1. Get the feature importances from the trained Random Forest model
5 importances = random_forest.feature_importances_
6 # 2. Create a bar plot to visualize them
7 feature_names = iris.feature_names
8 indices = np.argsort(importances)
9 plt.figure(figsize=(10, 6))
10 plt.title("Feature Importances from Random Forest")
11 plt.barh(range(len(indices)), importances[indices], color='b', align='center')
12 plt.yticks(range(len(indices)), [feature_names[i] for i in indices])
13 plt.xlabel('Relative Importance')
14 plt.show()
```



Knowledge CheckInstructions: Answer the following questions in this markdown cell.

- 1. In your own words, what is the main idea behind ensemble methods? Why is a "committee" of models often better than a single expert model?**

Ensemble methods combine multiple models into a "committee" to make a collective prediction rather than relying on a single model. This is built on the "wisdom of the crowd" individual models may make mistakes, overfit, or have blind spots, but when diverse models vote or average their predictions together, their individual errors cancel each other out. This produces a final result that is much more stable, accurate, and robust than any single expert model.
- 2. Which model performed better in this lab, the single Decision Tree or the Random Forest? Is this what you expected? Why?**

Both models achieved 100.00% accuracy on the test set. While you might initially expect the Random Forest to outperform a single Decision Tree because it prevents overfitting, the Iris dataset is small, clean, and has very distinct, linearly separable classes. Because of this, a single decision tree is powerful enough to achieve perfect separation on its own, though Random Forests remain much more reliable on larger, noisier datasets.
- 3. Looking at the feature importance plot, which two features were most important for classifying the iris flowers? Does this match what you might expect from your own knowledge of flowers?**

The two most important features for classifying the iris flowers are petal length and petal width. This aligns with botanical expectations because different iris species have distinct, easily measurable variations in their petals that make them far more reliable for classification than sepal dimensions.

1 Start coding or generate with AI.